부록 · Docker 샌드박스 · Compose

Docker Sandbox Codex SpringBoot Kotlin MariaDB SSH 초보자교본 (1)

Docker Sandbox + Codex AI Spring Boot Kotlin + MariaDB + SSH 초보자 실습 교본

YAML 중심의 통합 개발환경 구축 · 접속 · 빌드 · 실행 · DB 확인

이 교본은 Docker 기반 격리 개발환경에서 Kotlin + Spring Boot 애플리케이션, MariaDB, SSH 접속환경을 함께 구성하고, Codex AI를 이용해 코드를 수정한 뒤 clean → build → 실행하는 전체 흐름을 초보자 관점에서 정리함.

1. 완성 후 전체 구조

```bash
Windows / Linux 호스트
        |
        +-- SSH 2222 ----------> app 컨테이너 :22
        |
        +-- HTTP 8080 ---------> Spring Boot :8080
        |
        +-- DB 3307 -----------> MariaDB :3306
                                    ^
                                    |
                          mariadb:3306
                                    |
                          Spring Boot Kotlin
                                    |
                               Codex AI
```

핵심은 호스트에서 MariaDB에 접속할 때는 127.0.0.1:3307을 사용하고, app 컨테이너의 Spring Boot에서는 Docker 서비스 이름인 mariadb:3306을 사용한다는 점임.

2. 준비물

항목	용도
Docker Desktop 또는 Docker Engine	컨테이너 실행
Docker Compose	compose.yml 실행
SSH Client	가상 개발환경 접속
JDK 21	Spring Boot/Kotlin 실행
Gradle Wrapper	빌드
Codex AI	프로젝트 분석·코드 수정·명령 실행 지원
MariaDB	애플리케이션 데이터베이스

3. 프로젝트 폴더

```text
kotlin-sandbox/
├── compose.yml
└── app/
    ├── build.gradle.kts
    ├── settings.gradle.kts
    ├── gradlew
    ├── gradlew.bat
    ├── gradle/
    └── src/
        └── main/
            ├── kotlin/
            └── resources/
                └── application.yml
```

4. 전체 compose.yml

아래 예제는 학습용임. 실제 운영환경에서는 비밀번호를 YAML에 직접 기록하지 말고 .env 또는 secrets로 분리하는 것이 좋음.

```yaml
services:
  mariadb:
    image: mariadb:11
    container_name: sbx-mariadb
    restart: unless-stopped
    environment:
      MARIADB_ROOT_PASSWORD: root1234
      MARIADB_DATABASE: wizdb
      MARIADB_USER: devuser
      MARIADB_PASSWORD: CHANGE_ME
    ports:
      - "3307:3306"
    volumes:
      - mariadb_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mariadb-admin", "ping", "-h", "127.0.0.1",
             "-uroot", "-proot1234"]
      interval: 10s
      timeout: 5s
      retries: 10

  app:
    image: eclipse-temurin:21-jdk
    container_name: sbx-kotlin
    restart: unless-stopped
    working_dir: /workspace
    depends_on:
      mariadb:
        condition: service_healthy
    ports:
      - "8080:8080"
      - "2222:22"
    volumes:
      - ./app:/workspace
      - gradle_cache:/root/.gradle
    environment:
      SPRING_DATASOURCE_URL: jdbc:mariadb://mariadb:3306/wizdb
      SPRING_DATASOURCE_USERNAME: devuser
      SPRING_DATASOURCE_PASSWORD: CHANGE_ME
    command:
      - /bin/bash
      - -c
      - |
        apt-get update
        apt-get install -y openssh-server curl git vim nano \
          mariadb-client iputils-ping net-tools procps

        mkdir -p /run/sshd

        if ! id devuser >/dev/null 2>&1; then
          useradd -m -s /bin/bash devuser
        fi

        echo 'devuser:CHANGE_ME' | chpasswd

        sed -i 's/^#*PermitRootLogin.*/PermitRootLogin no/' \
          /etc/ssh/sshd_config
```

```
        sed -i 's/^#*PasswordAuthentication.*/PasswordAuthentication yes/' \
          /etc/ssh/sshd_config

        chown -R devuser:devuser /workspace || true
        /usr/sbin/sshd

        echo "Docker Kotlin Sandbox Ready"
        echo "SSH       : localhost:2222"
        echo "SpringBoot: localhost:8080"
        echo "MariaDB   : mariadb:3306"

        tail -f /dev/null

volumes:
  mariadb_data:
  gradle_cache:
```

5. 실행과 상태 확인

```bash
cd kotlin-sandbox
docker compose up -d
docker compose ps
```

문제가 있으면 로그를 확인함.

```bash
docker compose logs -f app
docker compose logs -f mariadb
```

6. SSH로 가상환경 접속

Docker가 실행되는 같은 PC에서 접속:

```bash
ssh -p 2222 devuser@127.0.0.1
```

비밀번호: CHANGE_ME

다른 LAN PC에서 Docker 호스트의 IP가 SERVER_IP인 경우:

```bash
ssh -p 2222 devuser@SERVER_IP
```

접속 후 확인:

```bash
whoami
pwd
java -version
cd /workspace
ls
```

7. Spring Boot application.yml

```yaml
server:
  port: 8080

spring:
  datasource:
    url: jdbc:mariadb://mariadb:3306/wizdb
    username: devuser
    password: CHANGE_ME
    driver-class-name: org.mariadb.jdbc.Driver

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

컨테이너 내부에서 localhost:3307을 사용하면 안 됨. Spring Boot와 MariaDB는 Docker 내부 네트워크에서 서비스명 mariadb와 내부 포트 3306으로 통신함.

8. build.gradle.kts 핵심 의존성

```kotlin
dependencies {
    implementation("org.springframework.boot:spring-boot-starter-web")
    implementation("org.springframework.boot:spring-boot-starter-data-jpa")
    implementation("org.jetbrains.kotlin:kotlin-reflect")
    runtimeOnly("org.mariadb.jdbc:mariadb-java-client")
    testImplementation("org.springframework.boot:spring-boot-starter-test")
}
```

9. MariaDB 직접 접속

호스트 PC에서:

```bash
mariadb -h 127.0.0.1 -P 3307 -u devuser -p
```

비밀번호: CHANGE_ME

app 컨테이너에서:

```bash
mariadb -h mariadb -P 3306 -u devuser -p
```

DB 확인 예:

```sql
SHOW DATABASES;
USE wizdb;
SHOW TABLES;
```

10. Spring Boot clean → build → 실행

```bash
ssh -p 2222 devuser@127.0.0.1
cd /workspace

chmod +x gradlew
./gradlew clean
./gradlew build
./gradlew bootRun
```

브라우저 또는 REST 클라이언트에서는 http://127.0.0.1:8080 으로 접근함.

11. Codex AI를 함께 사용하는 흐름

Codex AI는 프로젝트 폴더를 기준으로 소스 구조를 읽고 수정하거나, 빌드 오류를 분석하고 필요한 변경을 제안·적용하는 개발 보조 도구로 활용할 수 있음. Codex의 정확한 실행 명령과 인증 방식은 설치 버전에 따라 달라질 수 있으므로 현재 설치된 Codex CLI의 도움말을 먼저 확인하는 방식이 안전함.

```bash
cd /workspace

codex --help
codex
```

권장 작업 순서:

```text
1) SSH로 sbx-kotlin 접속
2) cd /workspace
3) Codex 실행
4) "Spring Boot Kotlin 프로젝트를 분석해줘"
5) "MariaDB JPA 연결 설정을 확인해줘"
6) "필요한 코드를 수정해줘"
7) Codex 종료 또는 별도 터미널 사용
8) ./gradlew clean
9) ./gradlew build
10) ./gradlew bootRun
11) 브라우저/REST API로 테스트
12) MariaDB에서 저장 결과 확인
```

Codex에게 전달할 초보자용 예시 지시문:

```text
이 프로젝트는 Kotlin + Spring Boot + JPA + MariaDB 프로젝트다.
DB 서비스명은 mariadb, 내부 포트는 3306이다.
기존 코드를 먼저 분석하고 오류가 있으면 수정해줘.
수정 후 ./gradlew clean 과 ./gradlew build 로 검증해줘.
실패하면 오류 원인을 설명하고 다시 수정해줘.
```

12. Codex까지 컨테이너에서 사용하려면

현재 compose.yml의 app 컨테이너는 JDK 중심 이미지이므로 Codex CLI가 기본 포함되어 있지는 않음. Codex CLI를 컨테이너 안에서 직접 실행하려면 해당 CLI가 요구하는 런타임/설치 절차와 인증정보를 추가해야 함. 반대로 호스트의 Codex에서 ./app 폴더를 열어 수정하고, SSH 터미널에서는 빌드·실행만 하는 구성도 단순하고 관리하기 쉬움.

즉 초보자에게는 다음 두 방식 중 ①을 먼저 권장함.

```text
① 호스트 Codex → ./app 소스 수정
   SSH → /workspace에서 clean/build/bootRun

② SSH → 컨테이너 내부 Codex 실행
   → 코드 수정 → clean/build/bootRun
   (Codex CLI 설치 및 인증 설정이 추가로 필요)
```

13. 포트 한눈에 보기

기능	호스트 포트	컨테이너 포트	접속 예
SSH	2222	22	ssh -p 2222 devuser@127.0.0.1
Spring Boot	8080	8080	http://127.0.0.1:8080
MariaDB	3307	3306	mariadb -h 127.0.0.1 -P 3307 ...

14. 자주 발생하는 오류

SSH 접속 거부:

```bash
docker compose ps로 app 실행 여부를 확인하고 docker compose logs app 및 컨테이너 내부 sshd 상태를 확인함.
```

DB 연결 실패: Spring Boot의 JDBC 주소가 jdbc:mariadb://mariadb:3306/wizdb인지 확인함. 컨테이너 내부에서 127.0.0.1:3307을 사용하지 않음.

gradlew 실행 권한 오류:

```bash
chmod +x gradlew 실행 후 다시 ./gradlew build를 실행함.
```

8080 포트 충돌: 호스트에서 이미 8080을 사용 중이면 compose.yml의 왼쪽 포트만 예: 8081:8080으로 변경함.

3307 포트 충돌: 호스트 포트를 예: 3308:3306으로 변경하고 호스트 DB 접속 시 -P 3308을 사용함.

15. 종료·재시작·초기화

```bash
# 종료
docker compose down

# 다시 시작
docker compose up -d

# 컨테이너 상태
docker compose ps

# DB 데이터까지 완전히 삭제하고 초기화
docker compose down -v
```

주의:

```bash
docker compose down -v는 MariaDB 볼륨 데이터까지 삭제하므로 필요한 DB가 있으면 실행하지 않음.
```

16. 최종 실습 체크리스트

- Docker 및 Compose가 정상 설치됨
- compose.yml로 MariaDB와 app 컨테이너가 실행됨
- ssh -p 2222 devuser@127.0.0.1 접속 성공
- /workspace에 Kotlin Spring Boot 프로젝트가 보임
- MariaDB 서비스명 mariadb:3306으로 연결됨
- ./gradlew clean 성공
- ./gradlew build 성공
- ./gradlew bootRun 성공
- 127.0.0.1:8080에서 API 접근 성공
- Codex AI로 소스 분석/수정 후 다시 빌드 검증함

17. 핵심 명령어 요약

```bash
docker compose up -d
docker compose ps

ssh -p 2222 devuser@127.0.0.1

cd /workspace
./gradlew clean
./gradlew build
./gradlew bootRun

mariadb -h 127.0.0.1 -P 3307 -u devuser -p

docker compose logs -f app
docker compose down
```

끝 - Docker + Kotlin + Spring Boot + MariaDB + SSH + Codex AI 통합 실습

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.